FridgeFit Implementation Report

“Cook More. Waste Less.”

40005 Advanced iOS Development
Assessment 4: App Store-Ready Production Solution

Team Number: FridgeFit
Students:
Yen-Chun Liu 26022020
Li-Ying Chen 25623618
Kai-Hsiang Chuang 25605484
Jinhao Feng 25744780
Shelly Huang 25725906

Table of Content

Section 1: Problem Statement, Stakeholders & Competitor Analysis....4
Team Identification....4
1.1 Problem Statement....4
1.2 Stakeholder Profile....5
Primary Stakeholders: University Students and Young Working Professionals....5
Secondary Stakeholders: Lifestyle-Specific and Budget-Conscious Meal Planners....5
1.3 Real-World Evidence and Problem Justification....7
1.3.1 Household Food Waste and Financial Impact....7
1.3.2 Dietary Habits and Balanced Eating....7
1.3.3 Product Opportunity....7
1.4 Competitor Analysis and Market Positioning....9
1.4.1 Competitor Comparison Matrix....9
1.4.2 Noms: Daily Food Diary & Log....10
1.4.3 Mob: Meal Planner and Recipes....10
1.4.4 ReciMe: Recipe Keeper....11
1.5 Mobile Platform Justification....12
Section 2 App Store Metadata & Product Strategy....12
2.1 App Store Connect Metadata....12
Third-Party Sign-In Compliance Note....13
2.2 Age Rating and Content Rights Justification....13
2.3 App Store Keywords....14
2.4 Promotional Text....14
2.5 Full App Store Description....14
2.6 Support and Marketing URLs....15
2.7 Privacy Policy Declaration and Privacy Nutrition Labels....16
A. Data Linked to You....16
B. Data Not Linked to You....17
C. Data Stored and Processed On-Device....17
D. Tracking and Advertising....18
E. iOS Permission Usage....18
F. Privacy Policy Statement....19
Section 3 System Architecture & Data Flow Diagram....20
Section 4 Team Contribution & Governance Matrix....21
4.1 Technical Role Breakdown....21

4.2 Deliverable Attribution Matrix 22
Contribution Verification 23
4.3 Git Contribution Evidence 24
Section 5 Group & Individual Reflective Report 27
Group Technical Reflection 27
Individual Reflection Statements 29
Yen-Chun Liu 29
Li-Ying Chen 29
Kai-Hsiang Chuang 30
Jinhao Feng 31
Shelly Huang 31

Section 1: Problem Statement, Stakeholders & Competitor Analysis

Team Identification

This report was prepared by the following five team members (full name and UTS student ID, matching the cover page):

Full Name	Student ID
Yen-Chun Liu	26022020
Li-Ying Chen	25623618
Kai-Hsiang Chuang	25605484
Jinhao Feng	25744780
Shelly Huang	25725906

1.1 Problem Statement

Food waste, meal-planning difficulties and inconsistent healthy eating habits remain significant challenges for Australian households, particularly among university students and young working professionals managing limited time and budgets. Although many individuals regularly purchase groceries and prepare meals at home, they may struggle to keep track of available ingredients, identify food approaching expiry and decide what to cook without purchasing additional items.

The underlying problem is not simply a lack of recipes but a disconnect between the ingredients people already own and the meals they can realistically prepare. Existing food applications commonly support recipe discovery, nutritional tracking or meal scheduling; however, these functions do not necessarily account for a user's changing household inventory, ingredient expiry dates and immediate lifestyle circumstances together.

For example, a student returning home after university may have chicken, spinach and rice available but lack the time or cooking experience to determine a suitable meal. A conventional recipe search may produce numerous options requiring additional groceries, while a food diary primarily records what has already been consumed.

FridgeFit addresses this gap by transforming household ingredient information into practical, personalised cooking recommendations. The proposed iOS application combines fridge and pantry inventory management, expiry awareness, recipe matching, dietary preferences and context-aware recommendations. Its FridgeFix AI enhancement feature further supports ingredient substitutions and recipe adaptation, helping users make meals with what they already have.

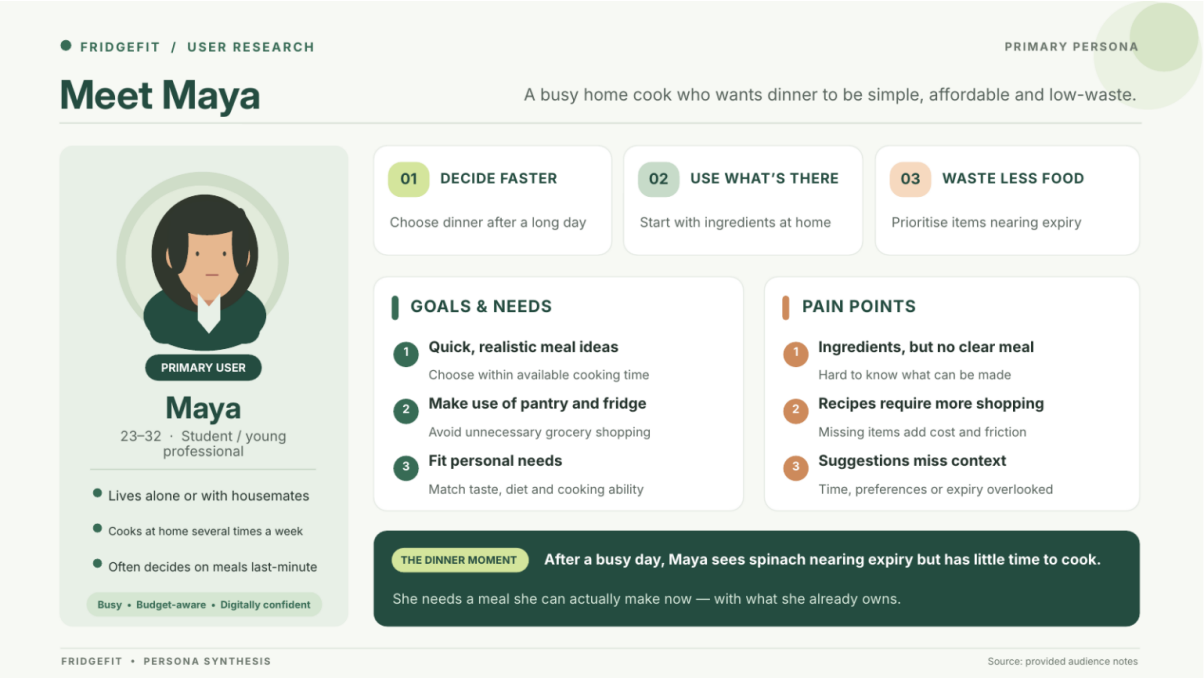
The project's objective is to reduce everyday cooking friction, unnecessary grocery purchases and avoidable food waste while encouraging convenient, balanced and enjoyable home cooking.

1.2 Stakeholder Profile

Primary Stakeholders: University Students and Young Working Professionals

FridgeFit primarily targets Australian university students and early-career professionals, approximately 18–35 years old, who live independently, with housemates or in small households, and prepare meals at home several times per week.

These individuals often balance academic or professional responsibilities with grocery shopping, cooking and household budgeting. Their limited time, variable schedules and developing cooking experience can make it difficult to use ingredients efficiently.



Note: This persona is a representative design scenario derived from the project's target-user definition, rather than a validated research participant.

Secondary Stakeholders: Lifestyle-Specific and Budget-Conscious Meal Planners

Secondary users include people with particular dietary preferences, fitness routines, work-from-home schedules, meal-preparation habits or sustainability objectives.

While these users share the primary group's need for practical recipe suggestions, they also require recommendations that reflect changing daily circumstances. For instance, a user

may prefer a quick meal during a working day, a more substantial meal following exercise or a low-waste recipe when several ingredients are approaching expiry.

FridgeFit addresses these requirements through optional lifestyle contexts, allowing users to prioritise cooking time, budget, meal preparation, dietary suitability or food-waste reduction without requiring restrictive fitness tracking.

Other relevant stakeholders include the development team responsible for application reliability and privacy, and recipe-content providers whose material may be imported subject to applicable usage rights.

1.3 Real-World Evidence and Problem Justification

1.3.1 Household Food Waste and Financial Impact

According to the Australian Government's Department of Climate Change, Energy, the Environment and Water (2025), Australia wastes approximately **7.6 million tonnes of food annually**, with households contributing around 2.5 million tonnes. The estimated total economic cost exceeds AUD $36.6 billion each year.

Research published by End Food Waste Australia (2025) further indicates that Australian households could potentially save approximately $2,500 annually through reductions in avoidable food waste.

These findings establish food waste as both an environmental and financial issue. They also support the need for practical household interventions that help consumers make better use of existing food.

FridgeFit responds directly to this problem through expiry-date tracking, ingredient prioritisation and recipe recommendations based on available inventory. Rather than relying solely on reminders, the app connects food-waste awareness with an actionable cooking decision.

1.3.2 Dietary Habits and Balanced Eating

The Australian Bureau of Statistics (2023), using data from the 2022 National Health Survey, reported that only **6.5% of Australian adults met the recommended daily vegetable intake**, while just 4.2% met both fruit and vegetable recommendations.

These figures demonstrate a substantial gap between recommended and reported dietary behaviours. Although insufficient vegetable consumption cannot be attributed solely to meal-planning difficulties, the findings reinforce the relevance of accessible tools that support balanced meal choices.

FridgeFit aims to encourage more varied, practical home cooking by recommending recipes aligned with ingredient availability, personal preferences and daily circumstances. The application is not intended to provide medical nutrition advice or substitute for professional dietary guidance.

1.3.3 Product Opportunity

The evidence identifies two related opportunities: helping households reduce preventable food waste and making balanced home cooking more manageable.

FridgeFit combines these objectives within a single workflow. By making ingredient availability and expiry information central to recipe selection, the proposed solution aims to turn food-waste awareness into immediate, achievable action.

The expected impact will require validation through usability testing and metrics such as recommendation relevance, additional ingredients required per recipe and successful inventory updates. National statistics establish the significance of the underlying problems but do not, by themselves, demonstrate the application's effectiveness.

1.4 Competitor Analysis and Market Positioning

Three applications were selected for comparison: **Noms: Daily Food Diary & Log**, **Mob: Meal Planner and Recipes**, and **ReciMe: Recipe Keeper**.

These products represent different approaches to food-related mobile experiences: food journaling, recipe-led meal planning and recipe organisation. Their existing capabilities help identify opportunities for FridgeFit to differentiate itself through inventory-aware meal recommendations.

1.4.1 Competitor Comparison Matrix

Competitor	Feature strengths	UX limitations relative to FridgeFit's use case	Business model limitations
Noms: Daily Food Diary & Log	Visual food journaling, AI food stickers, nutrition estimation, food-spending tracking and widgets	Primarily retrospective, centred on recording consumed meals rather than deciding what to cook with current inventory. Decorative and social interactions serve a different task from rapid ingredient-based decision-making	Freemium model with paid tiers; advertisements within its social experience may affect users seeking an uninterrupted experience
Mob: Meal Planner and Recipes	Chef-developed recipes, ingredient search, weekly meal planning, batch cooking and smart shopping lists	Strong recipe discovery, but its published workflow is more recipe- and planning-led than continuous tracking of fridge quantities and expiry dates	Premium subscriptions introduce recurring costs; some App Store reviews raise concerns about subscription and trial experiences
ReciMe: Recipe Keeper	Recipe imports from social media, personalised cookbooks, meal planning, shopping lists, serving adjustments and nutritional information	Optimised for collecting and organising recipes; its published features do not establish a complete inventory-to-expiry-to-recommendation workflow	Free version has restrictions, while unlimited imports and certain advanced capabilities require a premium subscription

Table 1: Matrix compared with Noms, Mob, and RecipMe

Sources: Apple App Store listings for Noms, Mob and ReciMe, accessed October 2026. UX limitations are comparative product assessments rather than independently measured usability defects.

1.4.2 Noms: Daily Food Diary & Log

Noms provides a visually engaging approach to food journaling through AI-generated food stickers, nutrition estimation, spending records and interactive widgets. Its emphasis on creative visualisation makes meal tracking more enjoyable and supports users interested in documenting dietary habits.

However, its experience is largely centred on **what users have already eaten**, rather than what they should prepare next. From FridgeFit's intended use case, this creates a functional mismatch: food logs and spending summaries do not directly resolve the problem of combining available ingredients into a practical recipe before food expires.

The playful interface and social features may also introduce interactions that are unnecessary for someone seeking a quick cooking recommendation, although these features are strengths for Noms' own audience.

Noms follows a freemium model with subscription and lifetime purchase options. Its social experience also includes advertising for non-paying users.

FridgeFit's differentiation: Shift the interaction from retrospective food tracking to proactive cooking decisions, using live pantry information and expiry-aware recommendations.

1.4.3 Mob: Meal Planner and Recipes

Mob is a more direct competitor because it supports recipe discovery, ingredient-based searching, weekly meal planning and automated shopping lists. Its chef-developed recipes and batch-cooking features provide a structured and appealing experience for users seeking culinary inspiration.

Its principal strength is helping users decide **which recipes to prepare and what groceries to purchase**. However, ingredient searching is not necessarily equivalent to maintaining a persistent record of current fridge quantities, storage locations and approaching expiry dates.

From FridgeFit's perspective, the opportunity is to make existing household inventory the starting point of the experience rather than requiring users to browse and plan recipes before determining ingredient availability.

Mob offers free access with premium subscription options. In the Australian App Store, listed premium purchases include monthly and annual plans, creating a potential cost consideration for budget-conscious users.

FridgeFit's differentiation: Prioritise the ingredients already owned, calculate pantry-match indicators and adapt recommended meals to changing daily contexts, thereby reducing the need for additional shopping.

1.4.4 ReciMe: Recipe Keeper

ReciMe specialises in recipe organisation and provides a convenient way to import cooking instructions from social media platforms, websites and other sources. It also supports digital cookbooks, grocery lists, weekly planning, nutrition information and ingredient scaling.

Its major advantage is consolidating recipes that would otherwise be scattered across multiple platforms. This significantly reduces the friction involved in saving and retrieving cooking inspiration.

Nevertheless, ReciMe's advertised functionality focuses primarily on recipe management and planning. Its published App Store description does not establish comprehensive tracking of household ingredient quantities, expiry risk and context-aware recipe ranking as one integrated workflow.

ReciMe operates a freemium subscription model, with premium access supporting unlimited social media recipe imports and additional functionality. This introduces an ongoing cost for users requiring extensive recipe organisation.

FridgeFit's differentiation: Extend beyond recipe storage by connecting saved recipes with actual pantry conditions, highlighting missing ingredients and recommending practical substitutions.

1.5 Mobile Platform Justification

A native iOS implementation is the definitive platform for FridgeFit, rather than a web or cross-platform alternative, because the application's value depends on three categories of device-level capability that a browser-based product cannot reliably provide.

On-device Neural Engine for private, offline AI. Ingredient recognition (Core ML, a fine-tuned YOLO model) and receipt interpretation and substitution suggestions (Apple's on-device Foundation Models) both run locally on the iPhone's Neural Engine. Photographs of a user's fridge and grocery receipts never leave the device, removing any dependency on server inference cost or latency, and the features work with no internet connection — directly relevant to a kitchen, often the weakest Wi-Fi spot in a home.

Offline-first local persistence. Checking what is in the fridge immediately before cooking is routinely performed without reliable connectivity. SwiftData stores the pantry, ingredient history and saved recipes entirely on-device, so the core workflow works with the phone in airplane mode or with no signal, which a thin web client cannot guarantee.

Camera and Local Network hardware for dual-device capture. Smart Scan uses the device camera and the Vision framework for live, real-time receipt and ingredient recognition, and Remote Camera uses WebRTC over the Local Network framework so a second iPhone signed in to the same account can act as a hands-free camera pointed at the fridge while the user's own phone shows the live recognition overlay — a peer-to-peer, low-latency capability specific to two pieces of Apple hardware on the same network, with no equivalent in a web app.

Taken together, these are specific, hardware-backed iOS capabilities — Neural Engine inference, on-device persistence, camera and Vision framework access, and peer-to-peer Local Network transport — that directly implement the time-pressured, often-offline, camera-centric use case described in Section 1.1.

Section 2 App Store Metadata & Product Strategy

2.1 App Store Connect Metadata

Field	Proposed App Store Submission
App Name	FridgeFit
Subtitle	Cook More. Waste Less.
Primary Category	Food & Drink
Secondary Category	Health & Fitness

Age Rating	Proposed 4+ (pending final App Store Connect assessment)
Pricing	Free
Minimum iOS Version	iOS 26.0

App Name Justification: FridgeFit is a concise, distinctive name combining the concepts of fridge management and finding suitable meals. It has nine characters, complying with Apple's 30-character limit.

Subtitle Justification: “Cook More. Waste Less.” conveys the application's primary value proposition: simplifying cooking decisions and reducing avoidable food waste. It contains 21 characters, complying with the 30-character limit.

Third-Party Sign-In Compliance Note

FridgeFit currently offers Firebase email/password sign-in and Google Sign-In, but does not implement Sign in with Apple. Apple's App Store Review Guideline 4.8 requires Sign in with Apple to be offered whenever another third-party login is available, so this is a known pre-production requirement rather than an oversight. Implementing it requires an active Apple Developer Program membership to configure the Sign In with Apple capability and register it as an identity provider in Firebase Authentication. As this assessment explicitly does not require teams to hold a paid Apple Developer Program account or to submit the app to Apple's review queue, the team deliberately scoped authentication to Firebase email/password and Google Sign-In for this submission. Adding Sign in with Apple is the team's identified final step before any real App Store listing.

2.2 Age Rating and Content Rights Justification

Proposed Age Rating: 4+

FridgeFit is a food-management and recipe-discovery application intended for general audiences. Its primary features involve maintaining pantry inventory, scanning grocery receipts, discovering recipes and receiving ingredient-substitution recommendations.

The application does not intentionally provide violent, sexual, gambling-related or otherwise mature content. Its main content-related considerations are:

- **User-generated content:** Users create personal pantry records and save recipes. The application does not provide a public social feed or unrestricted messaging feature.
- **Location sharing:** FridgeFit's core features do not require GPS tracking or geographic location sharing.
- **Health-related information:** Dietary preferences and intolerances may be applied to recipe filtering. FridgeFit does not diagnose medical conditions, prescribe treatment or claim to provide medical advice.

- **AI-generated content:** Apple's on-device Foundation Models support recipe suggestions, receipt interpretation and ingredient substitutions. Generated results are intended as cooking assistance rather than medical or professional nutritional guidance.

The proposed classification must be confirmed by completing Apple's current App Store Connect age-rating questionnaire against the final build.

2.3 App Store Keywords

Proposed keywords (comma-separated):

pantry,meal planner,ingredients,expiry,food waste,grocery tracker,receipt scan,substitutions

These keywords target users seeking assistance with ingredient organisation, food expiry, grocery scanning and meal preparation. They complement the application title and subtitle without repeating the brand name or including unrelated terms.

2.4 Promotional Text

Proposed promotional text:

Cook smarter with what you have. Scan groceries, track your pantry, discover ingredient-matched recipes and find clever substitutions with on-device AI.

2.5 Full App Store Description

FridgeFit - Cook More. Waste Less.

Ever opened your fridge and wondered what to cook?

FridgeFit helps you turn the ingredients you already have into delicious, practical meals. Whether you're a busy student, a young professional or simply trying to make your groceries last longer, FridgeFit makes everyday cooking easier.

ORGANISE YOUR FRIDGE

Keep track of the ingredients you own, including quantities, storage locations and expiry dates. Find what you need and identify ingredients that should be used soon.

DISCOVER RECIPES THAT FIT YOUR PANTRY

- Explore recipe ideas based on available ingredients.
- See which ingredients you already have and which are missing.
- Check ingredient availability before choosing a recipe.
- Save recipes for future cooking.
- Get AI-assisted recipe suggestions using your pantry information.

SCAN GROCERIES WITH EASE

Reduce manual entry using FridgeFit's scanning capabilities.

- Scan grocery receipts to extract ingredient information.
- Identify food items using camera-based recognition.
- Review recognised ingredients before adding them to your inventory.

Receipt interpretation combines Apple's Vision framework with on-device Apple Intelligence.

MISSING AN INGREDIENT? FRIDGEFIX IT.

Don't abandon a meal because you're missing one ingredient. FridgeFix uses Apple Intelligence to suggest ingredient alternatives and checks them against your pantry, helping you adapt recipes using ingredients you may already have.

COOK SMARTER, WASTE LESS

FridgeFit brings pantry organisation, recipe discovery and ingredient substitutions into one convenient iOS experience. Spend less time wondering what to cook, make better use of available groceries and simplify your everyday kitchen routine.

FridgeFit - Cook More. Waste Less.

2.6 Support and Marketing URLs

Required Resource	Proposed URL	Purpose
Support URL	`https://krischuang.github.io/FridgeFit/support.html`	Contact details, troubleshooting and FAQs
Marketing URL	`https://krischuang.github.io/FridgeFit/`	Product features, screenshots and app information
Privacy Policy URL	`https://krischuang.github.io/FridgeFit/privacy.html`	Data collection, storage, third-party processing and user rights
Project Documentation	https://github.com/krischuang/FridgeFit	Source repository and technical information
Walkthrough Video (Deliverable 3)	Watch FridgeFit Walkthrough Video	App demonstration video hosted on UTS OneDrive

The support and privacy policy URLs are live and deployed via GitHub Pages from the repository's website/ directory, and were verified reachable (HTTP 200) on 30 October 2026. The marketing URL reuses the same site's landing page; a dedicated marketing microsite can be added post-submission if required.

2.7 Privacy Policy Declaration and Privacy Nutrition Labels

FridgeFit uses a local-first architecture for core inventory management, while retaining third-party services for authentication, optional online recipes and cross-device connectivity.

The application's default persistence backend is **SwiftData**, which stores pantry inventory, activity history and saved recipes on-device. Firebase Authentication remains active for account management. Cloud Firestore supports remote-device registration and WebRTC signalling, and can serve as an alternative inventory backend.

Apple Intelligence features use on-device Foundation Models. The application also supports network-based Spoonacular searches when a valid API key is configured.

A. Data Linked to You

Data category	Purpose	Processing / storage
Email address	Firebase Authentication and Google Sign-In	Firebase/Google account services
User identifier	Account identification and separation of user records	Firebase Authentication and account-scoped Firestore records
Pantry content, if cloud backend is selected	Inventory management	Cloud Firestore
Remote-device details	Cross-device camera registration and availability	Cloud Firestore
WebRTC signalling metadata	Establish peer-to-peer camera sessions	Cloud Firestore

These categories are potentially associated with an authenticated user account. The repository's privacy manifest currently declares email address and other user content as linked to users for app functionality.

B. Data Not Linked to You

The repository does not currently establish a separate category of developer-collected data that can confidently be classified as permanently unlinked to users.

Therefore, no additional anonymous data-collection category is asserted in this draft. Third-party SDK diagnostics and any operational telemetry must be assessed before submitting the final App Store privacy questionnaire.

C. Data Stored and Processed On-Device

Data	Local technology	Purpose
Pantry items, quantities and expiry dates	SwiftData	Inventory management
Ingredient activity history	SwiftData	Tracking inventory changes
Saved recipes	SwiftData	Personal recipe collection
User presentation preferences	UserDefaults	Measurement units and serving defaults
Receipt photographs and recognised text	Vision + Foundation Models	On-device receipt interpretation
Camera imagery	Vision + Core ML	Ingredient recognition
AI prompts and generated suggestions	Foundation Models	On-device cooking assistance

Information that remains exclusively on-device is distinguished from information collected by the developer or third-party services.

When live Spoonacular searches are enabled, search terms such as ingredient names, dietary filtering preferences and a maximum cooking time (if the user has set one) may be transmitted to Spoonacular. The final disclosures must account for those requests and the provider's data practices.

D. Tracking and Advertising

FridgeFit's application privacy manifest declares `NSPrivacyTracking = false` and does not specify tracking domains.

No advertising-based cross-app tracking purpose has been identified in the application's own code.

The final declaration must also incorporate the behaviour and privacy manifests of Firebase, Google Sign-In and any other integrated third-party SDKs.

E. iOS Permission Usage

Permission key	Status	Usage justification
NSCameraUsageDescription	Declared	Camera access for fridge photography, ingredient recognition and optional cross-device camera streaming
NSPhotoLibraryUsageDescription	Not identified as necessary for current PhotosPicker workflow	System photo selection can occur without broad photo-library access
NSLocationWhenInUseUsageDescription	Not declared	FridgeFit does not require location tracking for its identified core features

NSFaceIDUsageDescription	Not declared	No Face ID-specific authentication flow is established
NSMicrophoneUsageDescription	Not declared	Confirmed not required: FridgeFit's WebRTC Camera Mode streams video only. Independent code audit (30 Oct 2026) found no AVAudioSession, microphone capture, or RTCAudioTrack usage anywhere in the codebase.

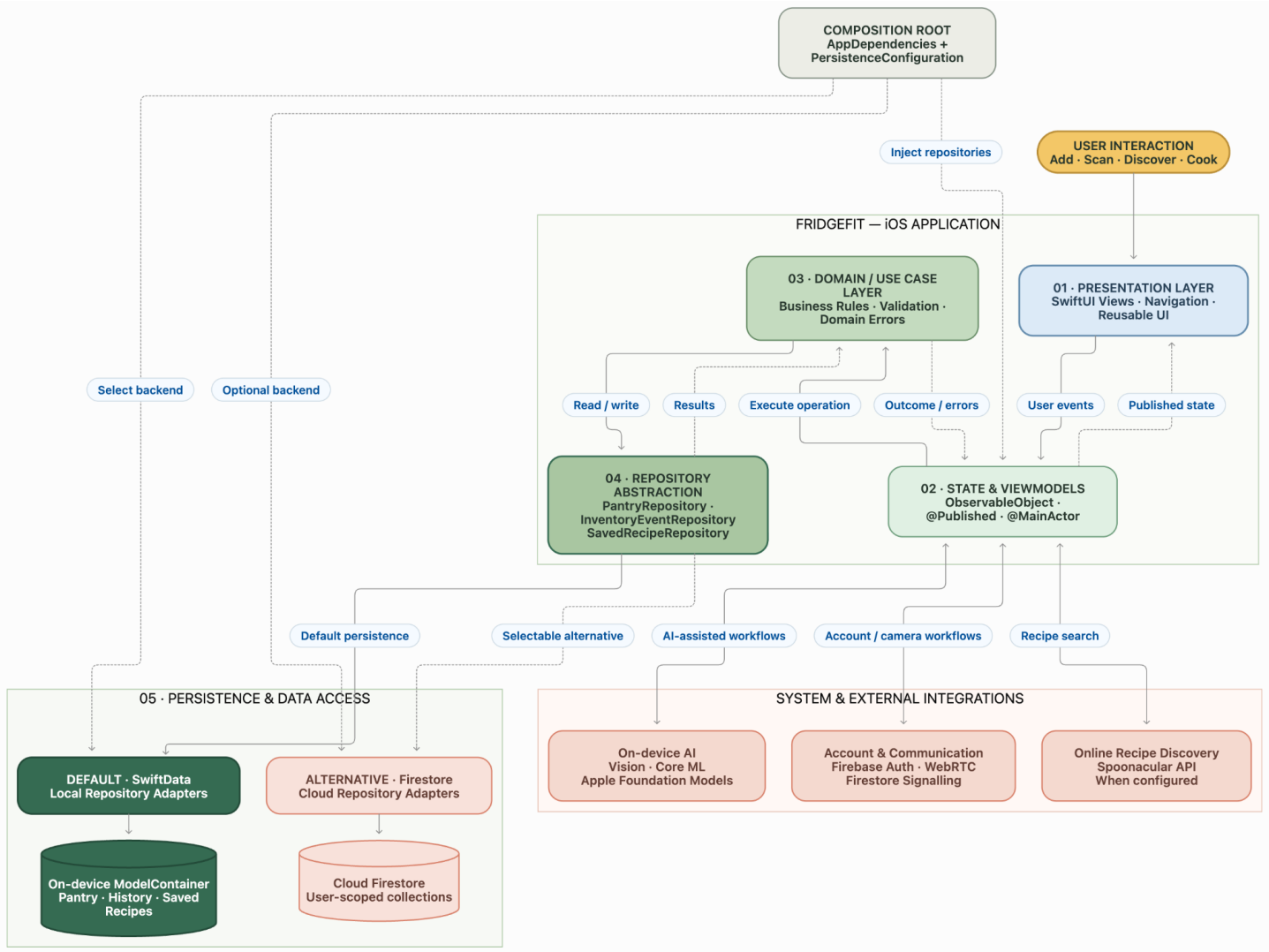
The current **`NSCameraUsageDescription`** explains that FridgeFit uses the camera to photograph fridge contents and, when Camera Mode is activated, share live video with another iPhone signed into the same FridgeFit account.

Permission requests should occur only when users access the relevant functionality. When camera access is denied, users should still be able to manage their ingredients manually.

F. Privacy Policy Statement

FridgeFit is designed to minimise unnecessary data transmission by keeping core inventory records on-device through SwiftData in its default configuration. AI-assisted receipt interpretation, recipe generation and ingredient substitutions use Apple's on-device Foundation Models framework. Camera-based recognition uses Vision and Core ML. FridgeFit's own privacy manifest declares that the application does not perform cross-app tracking. However, the final privacy policy and App Store declarations must accurately reflect the shipping configuration, third-party services, account data and any collected diagnostic information. The application should provide users with clear explanations of permission requirements, external services and available data-management options.

Section 3 System Architecture & Data Flow Diagram



Section 4 Team Contribution & Governance Matrix

4.1 Technical Role Breakdown

FridgeFit was developed collaboratively by a five-member team using distinct technical responsibilities across cloud infrastructure, platform integration, interface development, user experience and software testing. Responsibilities were allocated according to functional expertise, while GitHub branches and pull requests supported integration and collaborative review.

Team member	Assigned technical role	Area of responsibility
Kris (Kai-Hsiang)	Cloud and Extension Engineer	Cloud infrastructure, authentication, Firebase integration, backend connectivity and supporting platform services
Emily (Liying)	Extension Integration Engineer	Integration of extension-related and native iOS features, including coordination between device capabilities and application services
Selina (Yen-Chun)	UI/UX & HIG Specialist	SwiftUI interface design, navigation, recipe discovery experience, visual consistency, accessibility and Apple Human Interface Guidelines
Shelly	Frontend Developer, Verification QA	Development and integration of user-facing screens, interactive controls and application workflows, defect investigation and implementation validation
Jimmy (Jinhao)	Automation Testing and Verification QA	Automated testing, test-case development, regression verification, defect investigation and implementation validation

Although each member had a primary technical responsibility, several FridgeFit features required collaboration between interface, domain and integration layers. Accordingly, contributions should be assessed through implemented changes, pull requests, technical documentation and validation evidence rather than assigned roles alone.

4.2 Deliverable Attribution Matrix

FridgeFit was developed collaboratively by five team members, with responsibilities distributed across cloud infrastructure, platform integration, frontend development, UI/UX design and software verification.

Table 4.2 summarises each major development category, the features delivered, responsible contributors and representative Git commit evidence. Where multiple members contributed to a feature, the matrix acknowledges shared implementation and subsequent refinements.

Table 4.2. Consolidated Deliverable Attribution Matrix

Development Category	Features and Deliverables Included	Responsible Member(s)	Representative Git Commits
Authentication & Cloud Infrastructure	Firebase Authentication, Google Sign-In, Firestore repositories, user-scoped security rules and cloud connectivity	Kris (Lead), Jimmy (UI and rules refinements)	`9202de9b`, `ddf12914`, `172a3b00`
Persistence & Data Management	SwiftData persistence, repository adapters, inventory storage, saved recipe storage, ingredient history and backend configuration	Emily (Lead), Kris (original repositories), Jimmy (reliability improvements)	`94999a5b`, `29867484`, `20c6909f`
Inventory & Smart Scan	My Fridge, ingredient CRUD, quantity and expiry management, receipt scanning, ingredient recognition and confirmation workflows	Kris (Domain/AI), Shelly (Frontend), Jimmy (receipt reliability)	`2ed9add4`, `87f401e3`, `c4436af2`, `b34448f7`
Remote Camera & Platform Integration	Cross-device camera setup, device pairing, WebRTC communication, camera review and navigation	Kris (Backend), Shelly (UI), Jimmy (Integration fixes)	`e5a10693`, `25d66b3a`, `7bba6299`

Recipe Discovery & AI Integration	Recipe recommendation logic, Spoonacular integration, Discover filtering, saved recipes, recipe detail, Apple Intelligence substitutions and image caching	Kris (Domain/API), Emily (Data/AI), Selina (UI/UX), Jimmy (Performance)	`53de81dc`, `a95b42d0`, `308cae3c`, `cfef7a57`
UI/UX, Navigation & Accessibility	Home dashboard, Recipes navigation, Profile, reusable visual components, Fresh Botanical design system, Light/Dark Mode, HIG improvements and app icon	Selina (Lead), Shelly (Frontend), Jimmy (UI refinements)	`8b163c05`, `10be3d4a`, `e4c4f09d`, `ebf8da8a`
Impact Analysis & Price Tracking	Impact dashboard, food-waste insights, inventory valuation, ingredient price recording within My Fridge and related calculations	Jimmy (Logic/Integration), Selina (Initial UX), Shelly (My Fridge foundation)	`d2491e30`, `643e6556`, `e10b13d2`
Testing, Quality Assurance & Verification	Unit tests, repository contract tests, UI regression tests, Firestore rule verification, receipt reliability, Swift concurrency fixes and error handling	Jimmy (Assigned QA role), Kris, Emily, Selina (Verified testing contributions)	`49760b4c`, `398cb727`, `8023cb50`, `8e4b0968`
Report Documentation & Governance	Problem and competitor analysis, App Store metadata, architecture diagram, Git contribution verification, group reflection and signed individual reflections	All five members (collaborative); individual reflections signed and authored separately by each member in Section 5	Report drafts and final submission records

Contribution Verification

Individual technical contributions were verified against commits and pull requests in the FridgeFit GitHub repository. Attribution reflects identifiable implementation work, including original development, integration and subsequent refinements.

The final application incorporates financial and food-waste insights into Impact Analysis, with ingredient price entry accessible through My Fridge. Earlier FridgeFunds development is therefore represented under these integrated features rather than as a separate standalone module.

GitHub accounts used for verification were **`krischuang`** (Kris), **`imlililili`** (Emily), **`ycl2001`** (Selina), **`ShellyHZX`** (Shelly) and **`JimmyV0nn`** (Jimmy).

Full commit references and individual Git identities are summarised separately in Section 4.3.

4.3 Git Contribution Evidence

Table 4.3 summarises each member's commit history on the main branch. Counts exclude merge commits, because a merge commit records who merged a pull request rather than who wrote the code. Swift line changes are included only as an indication of volume: they do not measure effort or quality, and figures for test-heavy or design-focused work are not directly comparable with feature code.

Snapshot disclosure: this table reflects each member's commit history as recorded on 30 October 2026 and is not a live or self-updating record of the current HEAD of main. Commits made after that date — including the commit that adds this report to the repository — are not, and cannot be, reflected in the counts or hashes below.

Table 4.3. Git identities and commit summary (main branch, 30 October 2026)

Member	GitHub account	Commit author identities	Commits (excl. merges)	Active period	Swift lines (+/−)
Kris (Kai-Hsiang Chuang)	krischuang	krischuang, Kris	50	9 Sep – 23 Oct 2026	+53,693 / −2,576
Emily (Li-Ying Chen)	imlililili	imlililili, Li-Ying Chen	41	16 – 22 Oct 2026	+8,489 / −886
Selina (Yen-Chun Liu)	ycl2001	Yen-Chun Liu	30	18 – 23 Oct 2026	+5,691 / −1,190
Jimmy (Jinhao Feng)	JimmyV0nn	Jimmy	25	21 – 23 Oct 2026	+11,826 / −660

Shelly Huang	ShellyHZX	ShellyHZX, Shelly Huang	33	17 – 30 Oct 2026	+15,898 / −2,511
Total			**179**		

Table 4.4 lists representative commits for each member. Each commit can be opened in the repository by its abbreviated hash.

Table 4.4. Representative commits by member

Member	Commit	Description
Kris	`9202de9b`	Firebase Auth with email/password login and register screens
	`172a3b00`	Firestore security rules scoping the pantry to the signed-in user
	`4cf0deb2`	My Kitchen tab over the existing pantry
	`e5a10693`	Dual-device ingredient recognition from the WebRTC camera to the pantry
	`53de81dc`	Online recipe domain and API layer
	`257756f8`	Removed Firestore round trips from camera pairing
Emily	`94999a5b`	Relational SwiftData persistence
	`29867484`	Configurable persistence adapters
	`a95b42d0`	Ingredient substitutions migrated to Apple Foundation Models
	`b6dfb765`	Singular and plural ingredient names treated as the same ingredient
	`3060e3f1`	Saved recipe photos batched, decoded off the main thread and cached
Selina	`8b163c05`	Initial Home and recipe recommendation redesign

	`308cae3c`	Recipe discovery and filtering redesign
	`d4fcab0b`	Impact and waste dashboard
	`e4c4f09d`	Adaptive botanical colour tokens
	`ebf8da8a`	Adaptive contrast and accessible controls
Jimmy	`7093b92a`	Money domain for budgets, prices and waste
	`643e6556`	Budget and fridge-valuation use cases
	`d2491e30`	Impact & Budget summary assembled from pantry and budget data
	`b34448f7`	Verified persistent receipt recognition cache
	`c077fb79`	Shared value types kept off the main actor (concurrency fix)
	`1f43b689`	Sign-in and sign-up rebuilt on the Fresh Botanical design
Shelly	`10be3d4a`	FridgeFit UI design foundation
	`87f401e3`	My Fridge redesign and Smart Scan integration
	`c4436af2`	Smart Scan receipt flow redesign
	`25d66b3a`	Live AI camera setup and pairing UI
	`fb4bcfec`	Review screen opens automatically after remote analysis

Section 5 Group & Individual Reflective Report

Group Technical Reflection

The Impact & Budget screen passed its automated checks, but a new user could not enter a price for most groceries. We only noticed this when testing the app from the beginning. It was one of several problems that appeared after separate FridgeFit features were connected, especially where data moved between screens.

For persistence, the assignment required local storage, and saved recipes also needed to remain usable offline. We kept pantry records and saved recipes in SwiftData. Firebase Authentication manages email and Google sign-in, while Firestore supports account-related information, remote-device registration and WebRTC signalling. Discover had originally saved recipes through Firestore. Moving recipes and their photographs to SwiftData avoided another network request when opening an existing favourite and preserved access without internet. That choice left us with different data paths, which we had to describe accurately in the privacy material.

We organised the code using MVVM, domain use cases and repository interfaces so SwiftUI views were not responsible for storage operations. This was helpful when revising Remote Camera. Its screens relied on WebRTC and recognition services developed by other teammates; the navigation could be simplified while those services stayed intact. Integration was less tidy. One merge involved roughly 70 files, and shared dependency configuration required conflict resolution. Main-actor isolation produced concurrency errors in some value types too. Marking the relevant declarations nonisolated resolved those cases. Feature branches and smaller commits made it easier to see which changes belonged together.

A search result from Spoonacular was not necessarily a sensible recipe match. The API sometimes assumed staples such as sugar were available. Our matching logic had a separate flaw: sharing one word could make “ground pork” count as “ground beef”, or “cheddar cheese” as “cream cheese”. We changed the checks to work against actual pantry entries, including complete names, plurals and descriptive terms. Some recipe responses also contained missing instructions or images that would not load. Content cleaning, a blocklist and a hide option gave us ways to deal with those entries. The ranking also gives priority to ingredients nearing expiry, which made accurate pantry matching especially important for the recommendations we displayed.

API limits influenced how Discover works. Spoonacular's free plan allows 50 points per day, so a new request after every pantry edit was not reasonable. We instead recalculate matches for existing results on-device. Slow images needed a separate fix: batched database reads and image decoding outside the main thread.

Apple's App Review Guidelines influenced decisions in Smart Scan. Camera permission is tied to a camera task, and manual ingredient entry remains available to

someone who refuses access. Detection results go through Smart Scan Review before items enter My Fridge. A wrong name or quantity would otherwise remain in the inventory and affect later suggestions. We also removed unsupported gluten-free and allergen-safety claims from recipe features. Suggestions from a model can assist with cooking, but they cannot verify whether a substitution is safe for a particular allergy. Receipt scanning has fallback tests for unsuccessful recognition.

Data privacy was complicated because FridgeFit uses both Apple frameworks and outside services. We replaced OpenRouter-based ingredient substitution with on-device Apple Intelligence. Vision and Core ML support local recognition, while Firebase handles accounts, Firestore handles signalling and Spoonacular handles online recipe queries. Our proposed App Store disclosures distinguish those functions and their permission purposes. We also checked what users could do after refusing access or encountering an error, rather than assuming the ideal scan would always succeed.

The Fresh Botanical design system gave the team shared colours, typography and SwiftUI components. Light and Dark Mode work included adaptive tokens, contrast adjustments and simulator checks. Even with consistent screens, we had navigation decisions to make. FridgeFunds was originally planned as its own Budget tab. Six main tabs would have been crowded, so we placed its financial features in Profile as Impact & Budget. Remote Camera had a similar usability issue on a smaller scale. After “Analyse Ingredients”, users saw a captured-photo panel and then had to press another button to reach review. We removed the intermediate panel and redundant camera control. Successful analysis now opens Smart Scan Review directly.

The Impact & Budget issue was less visible. The team had more than 760 automated tests, including passing checks for the feature. However, prices could only be added through Value at Risk, which lists ingredients expiring within three days. Someone with a recently stocked fridge might see nothing there. We made every ingredient priceable from My Fridge. We also kept a manual test-case workbook after finding the problem, so testing covered ordinary navigation paths as well as specific functions.

SwiftData revealed another cross-feature effect. When a recipe save failed, rolling back the shared context could discard pantry edits that had not yet been saved. Saved recipes were given a separate context to stop those operations interfering with each other.

Receipt performance gave us a measurable result. Apple Intelligence initially needed about six to ten seconds to interpret a receipt. The verified cache uses a canonical image fingerprint: orientation and pixel representation are normalised before SHA-256 hashing, and only user-confirmed results are stored. In a repeated-scan test, processing fell from 5.91 seconds to roughly 16 milliseconds. This improvement applies to a cache hit; first-time recognition still requires processing. For two-iPhone pairing, we also reduced serial Firestore signalling work by moving session cleanup outside the critical path, combining session creation with the offer and starting ICE gathering earlier.

During integration we used code review, automated tests and manual checks, with ChatGPT, Codex and Claude Code assisting selected code inspection, frontend edits, test drafting and Git tasks. Suggestions were reviewed before being applied. For the final App Store-ready build, we paid particular attention to the paths that connect features. A recently

added ingredient can now have a price recorded in My Fridge, even when it is nowhere near expiry. The same attention went into keeping saved recipes accessible offline and making scanned ingredients available for review before storage.

Individual Reflection Statements

Yen-Chun Liu

I led FridgeFit's overall interface design, visual identity, and design system, working from the pain points identified in our competitor analysis and persona. In line with Apple's Human Interface Guidelines, my goal was a dependable, user-friendly interface. I created the Fresh Botanical design direction and turned it into consistent interaction patterns, colours, typography, and reusable elements throughout the app.

Alongside graphic design, I worked on SwiftUI frontend code, particularly organising user flows and navigation. I redesigned the Home dashboard, Discover, Saved Recipes, Recipe Detail, and Profile screens, and improved the navigation hierarchy between saved collections, recipe suggestions, and pantry-related elements.

A major challenge was maintaining consistency as teammates built features independently — differences in component styling, navigation, and layout sometimes disrupted the overall experience. I addressed this by enforcing a consistent interaction flow across features, introducing shared design rules, and improving frontend structures.

Implementing Light and Dark Mode while preserving accessibility was another challenge. I introduced adaptive colour tokens, adjusted contrast, and improved interactive controls for readability across appearances, using targeted UI testing and simulator validation to catch rendering and navigation problems.

Despite my prior UI/UX knowledge, FridgeFit was one of the few projects where I had to quickly turn design ideas into working SwiftUI interfaces. This pushed me beyond visual coherence to consider how use cases, navigation, and user flows interact. I learned that intuitive, maintainable interfaces need clearly defined user journeys aligned with application logic — and the project strengthened my ability to work across a multidisciplinary team, bridging UI/UX and frontend development.

Signed by: Yen-Chun Liu

Li-Ying Chen

My main responsibility in FridgeFit was online recipe matching. I integrated the Spoonacular API into the Discover feature, which recommends recipes from the user’s pantry. It ranks recipes that use soon-to-expire ingredients first, supporting our goal of reducing food waste. I also replaced the OpenRouter-based ingredient substitution with on-device Apple Intelligence. Finally, I implemented saving Discover recipes and their photos into SwiftData, so they stay available offline without using API quota.

The main challenge for me was that third-party data could not be trusted as-is. Spoonacular assumes staples such as sugar are always owned, so it overstated how many ingredients

the user had. Some recipes also arrived with missing steps or broken images. I resolved this by checking every ingredient against the user's actual pantry and handling plurals and descriptive words such as "mild". I also cleaned the provider's content and added a blocklist and a hide option for faulty recipes. The free plan's 50-point daily limit was a second constraint. Instead of calling the API again, I made the app re-check matches on the device whenever the pantry changes. When code review flagged slow image loading, I batched the database reads and moved image decoding off the main thread.

Working in a shared codebase taught me to keep each branch focused on one change, respond constructively to review, and prove behaviour with unit tests rather than assumptions. The assessment required local persistence, so I moved saved recipes from Firestore to SwiftData. I learned to turn an unreliable external service into a dependable feature while balancing correctness, cost and user experience.

Signed by: Li-Ying Chen

Kai-Hsiang Chuang

At the start of FridgeFit, I established the iOS project architecture and implemented much of the app's backend. I integrated Firebase Authentication with email/password and Google Sign-In, then configured Firestore security rules to restrict each user's data to their own account. I structured the pantry feature around domain models, use cases, repositories and view models, and extended these patterns to recipe feasibility, ingredient substitution, My Kitchen and recommendations. I also developed the dual-device WebRTC camera workflow and a receipt scanner to improve ingredient capture.

One of my biggest challenges was performance without sacrificing accuracy. Apple Intelligence initially took six to ten seconds to interpret a receipt. Rather than using less reliable recognition, I introduced a verified cache based on a canonical image fingerprint. By normalising orientation and pixel format before SHA-256 hashing, the cache recognises the same receipt even when its file representation changes. Cache entries are saved only after user confirmation. In testing, repeated scans dropped from 5.91 seconds to approximately 16 milliseconds.

Connecting two iPhones presented another challenge. The WebRTC signalling path involved sequential Firestore writes that delayed camera pairing. I moved session cleanup outside the critical path, combined session creation with the offer, and started ICE candidate gathering earlier to reduce setup latency.

As the team integrated features, I also addressed reliability and product integrity. I removed unsupported gluten-free and allergen claims, coordinated a merge affecting about 70 files, and helped expand our test suite beyond 760 cases. This project taught me that clear architecture, measurable optimisation and automated tests are essential when multiple developers contribute to the same app.

Signed by: Kai-Hsiang Chuang

Jinhao Feng

My role on FridgeFit was automation testing and verification QA — checking that each feature did what we claimed and fixing what I found. I also brought in FridgeFunds, a budgeting module I'd built earlier, merging it into Profile as Impact & Budget rather than adding a sixth tab. I wrote the money domain, budget and fridge-valuation use cases, and the combined pantry/budget screen.

Testing turned up problems that looked fine on the surface. In Discover, sharing one word counted as a match — “ground pork” made “ground beef” look owned — so I required full containment instead. In SwiftData, a failed recipe save rolled back the shared pantry context, discarding unsaved edits; giving saved recipes their own context fixed this. For receipt scanning, I added latency measurements and fallback tests, so the demo's speed claim was actually measured.

The defect I learned most from wasn't caught by any unit test: every Impact & Budget test passed, yet the screen never showed a dollar figure, since prices could only be added from the three-day Value at Risk list. I only noticed by using the app as a new user would, then made every ingredient priceable and kept a manual test-case workbook too.

Integration took longer than expected — main changed often, including the switch from Firestore to SwiftData, so I merged repeatedly and resolved conflicts in shared files like the dependency setup. Default main-actor isolation also caused concurrency errors in my value types until marked nonisolated.

I used Claude Code to read unfamiliar code, draft tests, and resolve merge conflicts, but still checked its suggestions against the app's real behaviour — some were wrong. My takeaway: passing tests aren't a working feature. Good QA tests the paths a real user takes, not just the code that's easiest to test.

Signed by: Jinhao Feng

Shelly Huang

Most of my work on FridgeFit was in SwiftUI. I built and refined parts of My Fridge, especially adding and editing ingredients, plus the way expiry dates are shown. Smart Scan, Profile, and the Remote Camera screens were also areas I worked on. Keeping everything close to our Fresh Botanical style mattered to me, although I cared just as much about whether the screens actually made sense to use.

The Remote Camera flow is probably the clearest example. After pressing "Analyse Ingredients", the user got a captured photo, then another panel, and only after that could open Smart Scan Review. It worked. Still, I thought it asked people to do too much for one task. Instead of touching WebRTC or the recognition code built by teammates, I changed the SwiftUI flow so a successful analysis opens Review directly. That also meant removing the extra Capture Photo button and the intermediate panel. Later, the standalone Open Camera option in Smart Scan went too. To me, having two similar ways to scan ingredients just made the screen harder to understand.

Meanwhile, our main branch kept changing as other features were integrated. Some updates needed checking carefully, particularly navigation changes and local Xcode settings that were only meant for device testing. Rather than adding everything at once, I kept the Remote Camera work in a feature branch and divided it into smaller commits. Easier to review, and less likely to interfere with what teammates were doing.

For the work itself, I used ChatGPT and Codex to help inspect existing code, plan focused edits, and handle Git steps. Those suggestions still needed checking against the app's intended behaviour. I now look at a screen differently. Not just whether it looks finished, but how many steps it makes someone take.

Signed by: Shelly Huang